Combinational Logic - Decoders

1. Overview of Encoders and Decoders

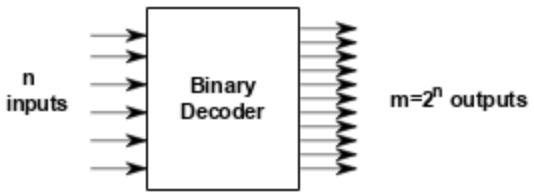
Encoders and decoders are foundational combinational modules used extensively in digital systems for data routing and compression.

- **Binary Decoders:** Convert an n-bit binary code into a single active output. They are built using AND/OR gate logic.
- **Binary Encoders:** Perform the reverse operation, converting one of 2^n inputs into an n -bit output. They are highly useful for data compression.

2. Binary Decoders

A decoder looks at its inputs, determines which binary number is present, and activates the one output corresponding to that number while keeping all other outputs inactive.

- In general, a decoder has N input lines and 2^N output lines.
- It is known as an n-to-m-line decoder (where $m = 2^n$).
- The basic decoding element is the AND gate, because it produces a logic 1 only when all inputs are 1.



The 2-to-4 Decoder

A 2-to-4 decoder has 2 inputs (A_1, A_0) and 4 outputs (D_3, D_2, D_1, D_0).

Truth Table:

A_1	A_0	D_0	D_1	D_2	D_3
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

Boolean Expressions (Minterms):

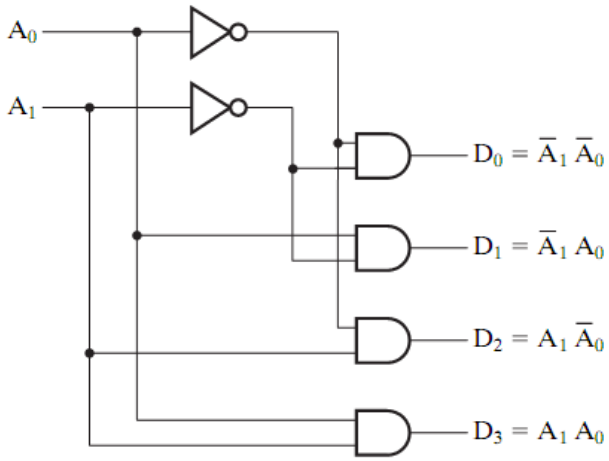
$$D_0 = \overline{A}_1 \overline{A}_0$$

$$D_1 = \overline{A}_1 A_0$$

$$D_2 = A_1 \overline{A}_0$$

$$D_3 = A_1 A_0$$

(Note: Each output represents exactly one 2-variable minterm).



Decoders with Enable (EN) Pin

Decoders often include an Enable pin.

- If $EN = 0$, the decoder is disabled and all outputs are forced to 0, regardless of the inputs.
- If $EN = 1$, the decoder functions normally according to its truth table.

Notice the use of "Don't Cares" (X) in the first row. Because $EN = 0$ kills the output, we don't care what A_1 and A_0 are.

EN	A_1	A_0	D_0	D_1	D_2	D_3
0	X	X	0	0	0	0
1	0	0	1	0	0	0
1	0	1	0	1	0	0
1	1	0	0	0	1	0
1	1	1	0	0	0	1

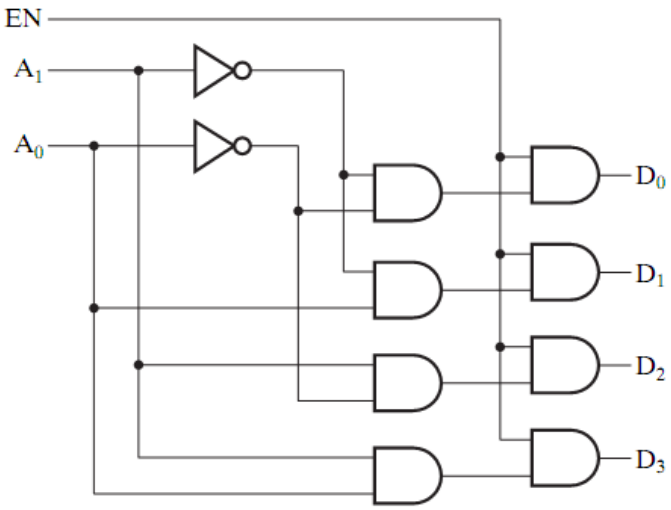
Boolean Expressions: Because the EN pin must be `1` for *any* output to trigger, it is simply ANDed (multiplied) into every single minterm equation.

$$D_0 = EN \cdot \overline{A}_1 \overline{A}_0$$

$$D_1 = EN \cdot \overline{A}_1 A_0$$

$$D_2 = EN \cdot A_1 \overline{A}_0$$

$$D_3 = EN \cdot A_1 A_0$$



The 3-to-8 Decoder

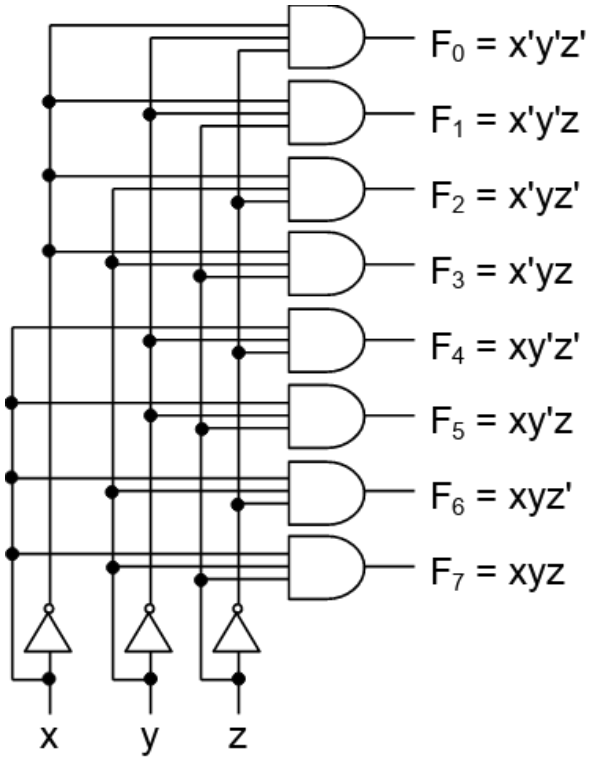
A 3-to-8 decoder takes 3 inputs (X, Y, Z) and generates 8 unique outputs (F_0 to F_7), corresponding to the 8 possible 3-variable minterms.

Truth Table:

X	Y	Z	F_0	F_1	F_2	F_3	F_4	F_5	F_6	F_7
0	0	0	1	0	0	0	0	0	0	0
0	0	1	0	1	0	0	0	0	0	0
0	1	0	0	0	1	0	0	0	0	0
0	1	1	0	0	0	1	0	0	0	0
1	0	0	0	0	0	0	1	0	0	0
1	0	1	0	0	0	0	0	1	0	0
1	1	0	0	0	0	0	0	0	1	0
1	1	1	0	0	0	0	0	0	0	1

Boolean Expressions:

- $F_0 = X'Y'Z'$
- $F_1 = X'Y'Z$
- $F_2 = X'YZ'$
- $F_3 = X'YZ$
- $F_4 = XY'Z'$
- $F_5 = XY'Z$
- $F_6 = XYZ'$
- $F_7 = XYZ$


F0 = x'y'z'
F1 = x'y'z
F2 = x'yz'
F3 = x'yz
F4 = xy'z'
F5 = xy'z
F6 = xyz'
F7 = xyz
x
y
z

3. Implementing Logic Functions Using Decoders

Any combinational circuit with n inputs and m outputs can be implemented using an n-to-2^n decoder and m OR gates.

- The decoder generates all possible minterms of the input variables.
- The OR gate simply sums (adds together) the specific minterm outputs required by the Boolean function.
- This approach is highly efficient when a circuit has many outputs, and each function is expressed with few minterms.

*Example: Implementing a Full Adder

The equations for a Full Adder are:

- Sum: $S(x, y, z) = \Sigma m(1, 2, 4, 7)$
- Carry: $C(x, y, z) = \Sigma m(3, 5, 6, 7)$

**Step 1: The Truth Table

A Full Adder adds three binary bits together: x (Input 1), y (Input 2), and z (Carry-In). It produces two outputs: S (Sum) and C (Carry-Out). We evaluate all 8 possible input combinations to determine the outputs.

Truth Table:

Minterm	x	y	z	S (Sum)	C (Carry)
m_0	0	0	0	0	0
m_1	0	0	1	1	0
m_2	0	1	0	1	0
m_3	0	1	1	0	1
m_4	1	0	0	1	0
m_5	1	0	1	0	1
m_6	1	1	0	0	1
m_7	1	1	1	1	1

**Step 2: Minterm Derivation

To use a decoder, we must express the outputs in standard **Sum of Minterms (SOP)** format. We do this by looking at the truth table and writing down the minterm index (m_i) everywhere an output is logic `1`.

For the Sum (S):

The output S is `1` at rows 1, 2, 4, and 7.

$$S(x, y, z) = \Sigma m(1, 2, 4, 7)$$

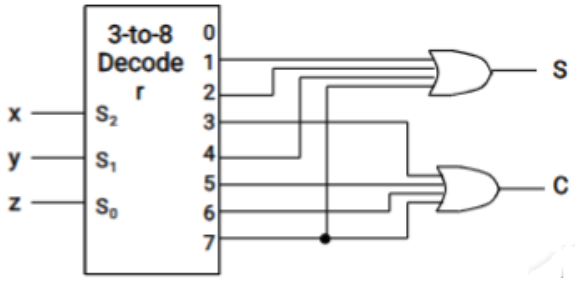
Expanded: $S = \overline{x}\overline{y}z + \overline{x}y\overline{z} + x\overline{y}\overline{z} + xyz$

For the Carry (C):

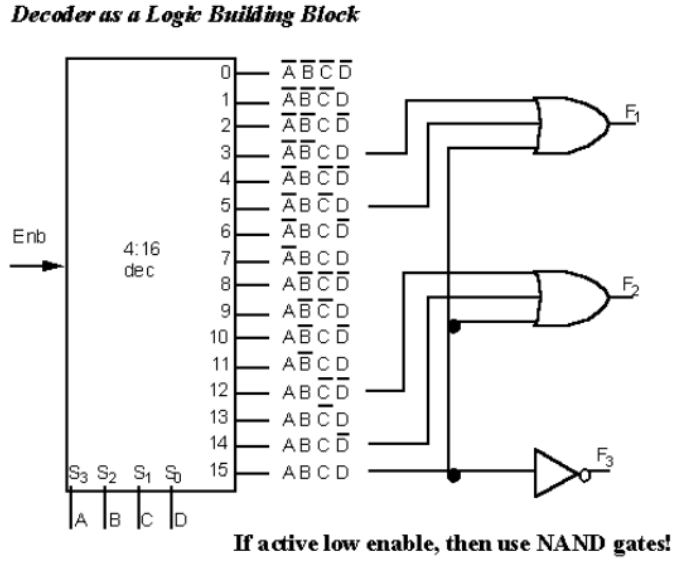
The output C is `1` at rows 3, 5, 6, and 7.

$$C(x, y, z) = \Sigma m(3, 5, 6, 7)$$

Expanded: $C = \overline{x}yz + x\overline{y}z + xy\overline{z} + xyz$



Example 2:



- **F1= A'B'CD + A'BC'D + ABCD**
- **solve the exercise for F_2 and F_3.

Solving for F_2

By tracing the three input wires of the F_2 OR gate backward to the 4:16 decoder outputs, we can identify its specific minterms:

- The top wire connects to **Pin 12**.
- The middle wire connects to **Pin 13**.
- The bottom wire connects to **Pin 14**.

Now, we just convert those pin numbers into their 4-variable Boolean equivalents based on the inputs A, B, C, and D (where A is the Most Significant Bit):

- Pin 12 (Binary `1100`) = $AB\overline{C}\,\overline{D}$
- Pin 13 (Binary `1101`) = $AB\overline{C}D$
- Pin 14 (Binary `1110`) = $ABC\overline{D}$

Since they feed into an OR gate, we add them together for the final expression:

$$F_2 = AB\overline{C}\,\overline{D} + AB\overline{C}D + ABC\overline{D}$$

**Solving for F_3

By tracing the single wire from the F_3 NOT gate (inverter) backward, we see it connects to the junction dot on **Pin 15**.

- Pin 15 (Binary `1111`) = $ABCD$

Because this signal passes through an inverter before reaching the output, the equation is the complement (NOT) of that minterm:

$$F_3 = (ABCD)'$$

If your professor prefers fully expanded expressions, you can apply De Morgan's Law to distribute the inversion:

$$F_3 = \overline{A} + \overline{B} + \overline{C} + \overline{D}$$

4. Decoder Expansion (Cascading)

You can build larger decoders using smaller ones by utilizing the Enable pin.

Example: Building a 4-to-16 Decoder using two 3-to-8 Decoders

- The 3 lower-order input bits (x, y, z) are wired into the data inputs of **both** 3-to-8 decoders.
- The 1 highest-order input bit (w) acts as the Master Select and is wired to the Enable pins.
- The w input is inverted before going into the top decoder. When $w = 0$, the top decoder is enabled (generating minterms 0-7), and the bottom is disabled.
- When $w = 1$, the enable conditions flip. The bottom decoder turns on and generates minterms 8-15.